Coda: Dynamic Priority and Traffic Interleaving for Multi-Tenant DLT Clusters

Anonymous Author(s)

ABSTRACT

The computational demands of deep learning training (DLT) are growing rapidly. To this end, cloud providers are developing large-scale clusters to host DLT jobs from multiple tenants. In such environments, inter-job communication contention has become a major challenge, severely degrading training performance and underscoring the need for efficient communication optimizers. Current approaches often fall short, suffering from either high rescheduling overhead due to traffic volatility or rigid scheduling due to the absence of global orchestration. To address these limitations, we propose Coda, a modular-yet-coordinated framework that mitigates contention through three core designs: (1) a congestion inertia mechanism that filters transient volatility locally to prevent frequent rescheduling; (2) a priority-aware global traffic interleaver that unifies isolated link-level traffic interleave schedule into a conflict-free global schedule, overcoming the rigidity of local decisions; and (3) a synergistic controller that operates within a unified optimization framework to make one-shot decisions, thereby balancing global scheduling optimality with local execution responsiveness. Our testbed and simulations using real production cluster traces demonstrate that Coda reduces average job completion time by up to 31% compared to existing methods.

1 INTRODUCTION

The increasing complexity of large models has driven model sizes to grow rapidly, with some now reaching trillions of parameters [31]. Training such models demands computational resources far beyond single GPU's capacity. To address this, various parallelism strategies have been proposed to distribute deep learning training (DLT) jobs across multiple GPUs [18, 25, 28]. While these parallel strategies scale training, it incurs significant communication overhead that scales with parallel sizes [11]. Consequently, communication has emerged as DLT's bottleneck [30], highlighting the need for optimization.

Existing solutions mostly concentrate on intra-job communication scheduling [10, 12, 13, 15, 20, 26], either by assigning flow priorities or determining flow routes within individual DLT job. Although these approaches prove effective for standalone scenarios, they struggle to accommodate large multi-tenant clusters due to their blindness to inter-job traffic contention [4]. Recently, major DLT service providers (*e.g.*, AWS, Google, and Alibaba) are increasingly deploying large-scale clusters to host DLT jobs from multiple tenants [21]. In these environments, the co-location of DLT jobs inevitably leads to the contention of network bandwidth resources which we refer to as *inter-job communication contention*. Traces from our multi-tenant cluster demonstrate that inter-job communication contention causes an average 40% increase in job completion time, even with the state-of-the-art intra-job communication scheduler [27]. Therefore, addressing inter-job communication contention has become critical for maintaining training efficiency in multi-tenant clusters.

Optimization of inter-job communication contention has recently focused on two approaches: traffic interleaving (*e.g.*, Cassini [23] and Symphony [3]) and priority assignment (*e.g.*, Crux [4], MLTCP [24] and PrioPlus [34]). For traffic interleaving, Cassini operates at the job granularity, adjusting the start times of DLT jobs to interleave their communication and computation phases. However, its reliance on precise traffic prediction makes this approach fragile. Runtime unpredictability, such as iteration time jitter [33] or failures [7], frequently disrupts the schedule, necessitating high-overhead rescheduling. To improve robustness of interleaving, Symphony operates at the finer granularity of collective operators. This improvement comes at a cost: to keep scheduling overhead low, Symphony ignores small operators, allowing them to fair-share the bandwidth with other operators. This reveals a fundamental trade-off inherent to traffic interleaving methods: a choice between the high overhead of frequent rescheduling and the compromised efficiency from incomplete interleaving.

Conversely, priority-based approaches offer a more adaptive solution, bypassing the need for precise traffic pattern prediction. Crux [4], for instance, introduces the concept of GPU intensity to quantify a job's computation to communication ratio. It maps this metric to physical network priorities (*e.g.*, PFC queues) to prioritize computation-intensive jobs during contention, thereby aiming to maximize overall GPU utilization. However, we argue that Crux's effectiveness is fundamentally undermined by two limitations. First, the *scarcity of physical priority queues* (*e.g.*, 8 for PFC) severely restricts scheduling granularity. This limitation forces distinct DLT jobs to share the same flow priority. Moreover, due to the limited priority space, jobs are easily relegated to the lowest priority levels, making them vulnerable to severe bandwidth starvation. Second, and more fundamentally,

Crux suffers from a *static and rigid priority assignment*. Since priorities are fixed, low-priority jobs face persistent starvation without relief. Furthermore, Crux's purely decentralized and reactive nature makes it oblivious to job-level progress, leaving it incapable of detecting or mitigating such starvation. As shown in Figure 3, our reproduction of Crux on an 8-GPU testbed reveals that although it reduces average Job Completion Time (JCT), its rigid and uncoordinated design significantly degrades *tail* JCT. Although subsequent works like MLTCP [24] and PrioPlus [34] attempt to mitigate the queue scarcity issue via virtualization, they still fail to resolve the fundamental absence of proactive cluster-wide coordination.

Given the complementary limitations of traffic interleaving and priority-based approaches, a naive combination would seem promising. However, such a combination faces three fundamental challenges: (1) *Conflicting Scheduling Mechanisms:* Traffic interleaving relies on precise *temporal separation*, whereas static priorities enforce *strict prioritization*. When inevitable jitter [7, 33] causes overlaps, static priorities aggressively starve low-priority jobs, destroying the delicate timing required by interleaving. Thus, static priorities accelerate schedule degradation rather than preserving it. (2) *Persistent Coordination Overhead:* Ideally, local priorities should absorb volatility to minimize expensive global rescheduling. However, uncoordinated static priorities cannot actively correct schedule drifts. Consequently, the central scheduler must still intervene frequently to fix misalignment, meaning the system retains the prohibitive coordination overhead of pure interleaving. (3) *Algorithmic Complexity in Multi-Tenancy:* Coordinating DLT flows with cyclic link dependencies is NP-hard. Existing heuristics (like BFS in Cassini [23]) fail to converge in loops, while heavy-weight solvers are too slow for runtime adaptation. To our knowledge, no existing work successfully harmonizes these conflicting objectives in multi-tenant DLT clusters.

To resolve these challenges, we propose Coda, a modular-yet-coordinated contention optimization framework. Coda leverages a unique symbiosis: it uses global interleaving to minimize baseline contention and dynamic priority to absorb runtime volatility. Coda comprises three core designs: (1) *Dynamic Priority via Congestion Inertia.* Instead of rigid hardware queues or slow-converging soft priorities, we introduce a host-based window holding mechanism. By assigning a criticality factor to jobs, Coda allows high-priority flows to temporarily ignore congestion signals (creating congestion inertia) while freezing their window growth. This mechanism acts as a temporal shield, masking transient jitter for short flows and ensuring weighted fairness for long flows without inducing starvation (Section 3.2). (2) *Priority-aware Global Interleaving.* To resolve the circular dependencies that plague existing interleaving heuristics, we propose a priority-aware spanning tree unification algorithm. This component first calculates optimal time shifts locally on each link and then constructs a global synchronization spanning tree rooted in high-criticality jobs. This approach effectively breaks dependency cycles and unifies conflicting local schedules into a consistent global plan with low computational complexity (Section 3.4). (3) *Unified One-Shot Coordination.* Coda operates on a unified optimization framework where the central controller co-optimizes job-level time shifts and fine-grained priorities. Unlike systems that require continuous rescheduling, Coda makes a single, holistic scheduling decision upon job arrival. The global interleaving plan sets the coarse-grained schedule, while the local priority control mechanism automatically absorbs runtime deviations. This design eliminates the high overhead of continuous coordination while maintaining superior robustness. The design of Coda enables it to cooperate with popular job resource schedulers [6, 8, 9, 14, 16, 19, 22, 35], requiring minimal modifications. Our key contributions include:

(1) We conduct an in-depth analysis of production cluster trace, revealing that circular dependencies are widespread, complicating traffic interleaving heuristic design. We also show that existing static priority assignment approach inevitably degrades tail JCT due to bandwidth starvation.
(2) We design Coda, the first system to harmonize traffic interleaving with priority control. It features a novel *Window Holding* congestion control to absorb jitter and a spanning tree algorithm to resolve global circular dependencies. This coordinated design eliminates scheduling incoherence and ensures robustness against cluster volatility without high rescheduling overheads.
(3) We evaluate Coda through extensive experiments. In severe contention scenarios, Coda acts robustly, reducing job completion time by up to 31%. In production trace simulations, it outperforms state-of-the-art solutions, reducing average JCT by up to 41% while accelerating 95th-percentile iteration time by 1.5×.

2 BACKGROUND AND CHALLENGES

2.1 Background

To accommodate scaling model sizes, parallelism strategies partition training workloads across multiple GPUs. Data parallelism (DP) replicates the model to process data subsets, requiring global synchronization of gradients. Tensor parallelism (TP) [28] splits individual operators across devices, necessitating frequent communication of intermediate results. Pipeline parallelism (PP) [18] partitions layers into stages, introducing sequential dependencies for transferring activations. Modern training often combines these strategies

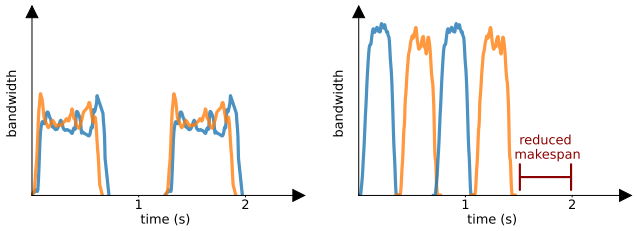


Figure 1: (a): Communication contention between two Llama-3.2-1B training jobs using distributed data parallel (DDP), (b): Resolve traffic contention by traffic interleaving.

(3D parallelism), generating massive, heterogeneous communication overheads that contend for network bandwidth.

Much attention has been paid to reducing communication overhead in distributed training through intra-job scheduling [10, 12, 13, 15, 20, 26, 27]. However, optimizing *inter-job* communication contention remains largely underexplored, despite its growing criticality in large-scale multi-tenant environments. When multiple jobs co-locate on shared infrastructure, they inevitably compete for network bandwidth on bottleneck links. Figure 1(a) illustrates this phenomenon, where bandwidth contention between two Llama-3.2-1B training jobs slows down communication, directly delaying job completion. Our traces collected from a production multi-tenant cluster reveal that inter-job communication contention leads to an average 40% increase in job completion time, even when state-of-the-art intra-job schedulers like MSCCL++ [27] are deployed.

To address inter-job communication contention, existing solutions primarily focus on two methodologies: *traffic interleaving* (*e.g.*, Cassini [23] and Symphony [3]) and *priority assignment* (*e.g.*, Crux [4], MLTCP [24], and PrioPlus [34]). However, we demonstrate in the following that both approaches exhibit fundamental limitations regarding runtime robustness and global coordination.

2.2 Limits of Job Priority Assignment

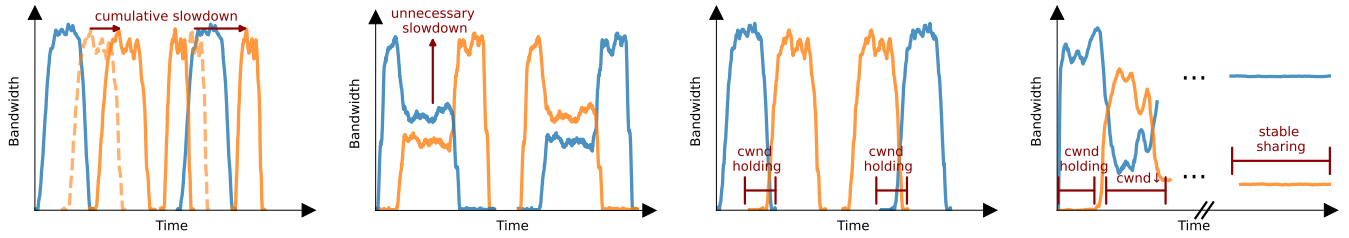
Crux introduces the concept of GPU intensity to quantify the computation-to-communication ratio of DLT jobs. The GPU intensity of job j is defined as: $I_j = \frac{W_j}{t_j}$, where W_j denotes the computation workload and t_j represents the maximum transmission delay. Crux maps this metric to static physical network priorities (*e.g.*, PFC queues), ensuring that computation-intensive jobs are prioritized during network contention.

However, we argue that Crux's priority-based scheduling has inherent limitations. First, Crux employs a static and rigid priority assignment. Priorities are fixed at job deployment based on profiling and remain unchanged during runtime. Combined with the limited number of hardware priority queues (*e.g.*, 8 for PFC), this rigidity forces low-priority jobs to suffer from persistent bandwidth starvation whenever they contend with high-priority traffic. Unlike dynamic systems that might temporarily boost a starving low-priority job, Crux's static policy leads to accumulating delays for low-priority jobs, as demonstrated in Figure 2(a). The cumulative delays eventually lead to degrade in tail Job Completion Time (JCT).

Second, and more fundamentally, Crux is constrained by its reactive, decentralized nature. By operating solely as a link-level mechanism, it can only mitigate contention locally at switches *after* packets have already queued up. It lacks the global visibility required to *proactively* schedule traffic or adjust job start times to eliminate the contention source in the first place. Consequently, while this design is robust to jitter, it fails to achieve the contention-free performance that global planning (like traffic interleaving) could theoretically offer. As illustrated in Figure. 3, we replicated Crux's priority control mechanism on our 8-GPU testbed, using a workload of 100 DLT jobs with a specified arrival pattern. Our experimental results validate that while Crux reduces the average JCT, its rigid and purely reactive design leads to significant degradation in tail JCT.

In contrast to the static, hardware-bound approach of Crux, MLTCP [24] proposes a flexible, host-based priority mechanism at the transport layer. MLTCP modifies the congestion control algorithm by scaling the congestion window (*cwnd*) increase rate based on the flow's progress within a training iteration. Specifically, it calculates a multiplier derived from the ratio of transmitted bytes to the total bytes of the current iteration. This allows flows nearing completion to increase their *cwnd* more aggressively, thereby preempting bandwidth to finish transmission and theoretically inducing automatic traffic interleaving.

However, we argue that this mechanism offers only a *soft priority*. Unlike strict hardware prioritization, MLTCP cannot shield high-priority flows from congestion (Figure 2(b)). When the switch buffer overflows, all flows, regardless of their progress, inevitably suffer from packet drops or ECN marks and must reduce their windows. The priority mechanism only takes effect during the window recovery phase *after* congestion has already occurred. This reactive nature leads to *slow convergence*, where the system often requires dozens of iterations to reach a stable interleaved state. PrioPlus [34], on the other hand, achieved strict virtual priority that enlarged the limited hardware priority space. However, it is designed for general data center workloads, therefore lacking the necessary domain-specific flexibility to effectively schedule periodic DLT traffic.



(a) Crux (b) MLTCP (c) CODA-PA for DLT flows (d) CODA-PA for elephent flows

Figure 2: The behavior of DLT flows under different priority policies.

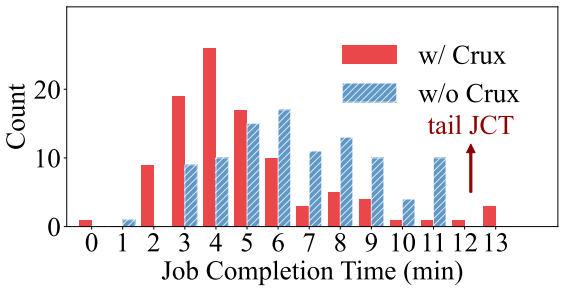


Figure 3: Job completion time (JCT) distribution with and without Crux.

2.3 Limits of Interleaving Job Traffic

Approaches like Cassini [23] exploit the periodic nature of DLT traffic, where communication (up) and computation (down) phases alternate. The core idea is to interleave these phases across jobs sharing a bottleneck link by introducing a precise time shift to one job's start time (Figure 1(b)). Theoretically, if traffic patterns remain perfectly stable, this achieves zero contention.

However, we argue that this approach faces a fundamental trade-off between robustness and scheduling overhead. First, traffic interleaving is inherently fragile due to runtime unpredictability. As mentioned in Section 1, production environments suffer from iteration time jitter [33] and device failures [7]. These variances disrupt the precise alignment required for interleaving, causing pre-calculated schedules to drift and contention to re-emerge instantly. To maintain efficiency, the scheduler must frequently recalculate and re-apply time shifts, incurring prohibitive coordination overhead that negates the performance gains.

Second, finding a valid global schedule is algorithmically challenging in multi-tenant clusters. Cassini attempts to unify local link-level decisions using a bipartite affinity graph and a Breadth-First Search (BFS) traversal. However, this heuristic fails when facing circular dependencies, a scenario appearing in 80% of our production contention cases. As shown in Figure 4(a), when three jobs compete in a cycle (Job $j_1 \rightarrow j_2 \rightarrow j_3 \rightarrow j_1$), their dependencies form a closed loop (Figure 4(b)). A simple traversal like BFS cannot resolve the conflicting time-shift constraints in such loops. Solving these complex dependencies requires heavy-weight solvers, further exacerbating the scheduling latency and making real-time adaptation to jitter impractical.

Symphony [3] attempts to improve robustness by shifting the scheduling granularity from entire jobs to individual collective operators. While this finer granularity allows for more flexible adjustments against jitter, it significantly expands the scheduling solution space. To prevent the coordination overhead from becoming prohibitive, Symphony is forced to compromise by ignoring small operators, leaving them to fair share bandwidth with large operators. This limitation results in *incomplete interleaving*: while large operators may be coordinated, the accumulated contention from unmanaged small operators continues to degrade network performance, confirming the inherent difficulty of achieving both low overhead and perfect isolation in interleaving-based approaches.

2.4 Our Intuition

Our key insight is that traffic interleaving and priority control are fundamentally complementary provided their inherent conflicts are resolved through unified coordination. Traffic interleaving excels at proactive global planning, structurally reducing contention by separating flows in time, but it is brittle against runtime jitter. Conversely, priority control excels at reactive adaptation, enforcing order during unexpected overlaps, but lacks the global view to prevent contention in complex topologies.

Therefore, CODA harmonizes these approaches under a unified optimization framework. Instead of running two conflicting scheduling loops, our central controller co-optimizes job-level time shifts and fine-grained priorities in a single, one-shot decision. In this symbiosis: (1) the interleaving policy creates a coarse-grained global schedule to minimize baseline contention; and (2) the dynamic priority mechanism acts as a robust execution layer, automatically absorbing runtime

jitter and enforcing the schedule without requiring the controller to perform high-overhead rescheduling. This design effectively resolves the deadlocks found in naive combinations and enables seamless integration with existing cluster schedulers [6, 8, 9, 14, 16, 19, 22, 35].

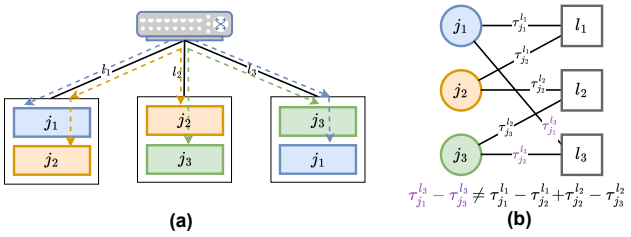


Figure 4: (a): Three DLT jobs with circular dependency. (b): Bipartite graph representation.

3 CODA DESIGN

3.1 System Overview

Figure 5 illustrates the architecture of CODA. Tenants submit DLT job requests to CODA's Priority Assignment (PA) engine. The PA engine first profiles each DLT job and then assigns priorities (job criticality factors) based on the profiling results (Section 3.2). Leveraging a decoupled architecture, the PA engine is designed to seamlessly integrate with existing GPU job schedulers, requiring minimal modifications. DLT jobs are subsequently deployed into the cluster based on the PA engine's priority assignment plan, potentially together with the job scheduler's deployment plan. The Traffic Interleaving (TI) engine obtains link status from the cluster. It first performs link-level traffic interleaving (Section 3.3) and then passes the results to the cluster-level interleaver for time shift unification (Section 3.4). The resulting unified interleave plan is sent to CODA's controller. The controller continuously monitors the global cluster status, enabling it to generate rapid one-shot schedules immediately upon the arrival of new DLT jobs.

3.2 Priority Control: CODA-PA

To address the rigidity of hardware queues and the weak isolation of soft priorities, we propose CODA-PA, a priority control framework that decouples priority into a global *static criticality* and a local *congestion inertia* mechanism.

To integrate global job context into our congestion control mechanism, CODA assigns a *job criticality factor* (β_j) to every job at deployment. This factor is grounded in the first principles of distributed training, capturing two key dimensions dictating a DLT job j's sensitivity to network performance:

(i) *Global Communication Scale*: Given a per-worker communication volume V_j, the total cluster-wide communication volume scales with the job's parallelism $\mathcal{S}_j$ (*i.e.*, the number of workers). This scaling relationship is modeled by a factor $\eta(\mathcal{S}_j)$, which accounts for the specific collective communication pattern (*e.g.*, ring or tree).

(ii) *Computation Time* (τ_j^{comp}): This metric reflects the job's tolerance to network delays. Jobs with short computation intervals τ_j^{comp} (*e.g.*, small-batch training) are typically communication-bound and suffer disproportionately from bandwidth fluctuations.

Combining these dimensions, we define the criticality factor as:

$$\beta_j = \eta(\mathcal{S}_j) \cdot \frac{V_j}{\tau_j^{\text{comp}}} \tag{1}$$

where both τ_j^{comp} and V_j are measured during a short profiling phase. Crucially, β_j depends solely on intrinsic job characteristics, ensuring consistent prioritization across varying cluster states while remaining computable at deployment time. Once assigned upon job deployment, β_j remains constant throughout the job's lifecycle, without requiring frequent control plane updates.

At the host side, CODA-PA translates β_j into a dynamic *window holding* mechanism. The core insight is to introduce "inertia" to the congestion control loop. As outlined in Algorithm 1, when a flow encounters congestion signals (*e.g.*,

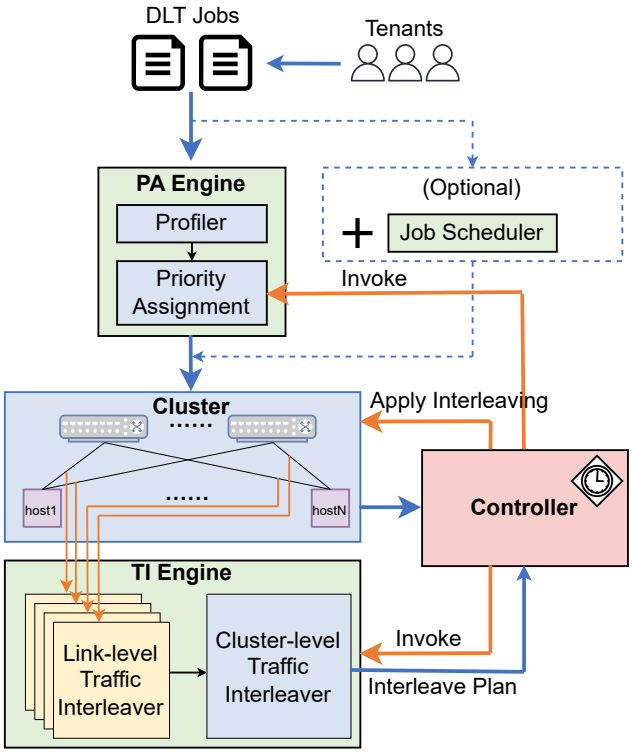


Figure 5: CODA system overview.

Algorithm 1: CODA-PA: Priority-based Window Holding

```
Input: ack, ecn, cwnd, β_j
Data: state ∈ {INCREASE, HOLDING}, timer
1  Function OnAck(ack, ecn):
2      t_now ← CurrentTime()
       // 1. Check Holding Expiration
3      if state == HOLDING and t_now > timer then
4          cwnd ← cwnd · (1 − α/2)   // Delayed Backoff
5          state ← INCREASE
6          return
7      end
       // 2. Handle Congestion Signal
8      if ecn == 1 then
9          if state ≠ HOLDING then
10             T_hold ← CalcDuration(β_j)
11             timer ← t_now + T_hold
12             state ← HOLDING
13         end
           // If Holding: Freeze cwnd (Ignore ECN)
14     else
           // 3. Standard Increase
15         if state == INCREASE then
16             cwnd ← cwnd + 1/cwnd
17         end
18         else
               // Holding: Freeze cwnd (No Increase)
19         end
20     end
```

ECN marking), it does not immediately reduce its congestion window (*cwnd*). Instead, the flow calculates a holding duration T_{hold} based on its criticality (line 10) and enters a holding state (line 12). The duration is bounded by the iteration time T_{iter}:

$$T_{hold} = \min\left(\kappa \cdot \beta_j \cdot \mathrm{RTT},\ T_{iter}\right) \tag{2}$$

where κ is a scaling coefficient (more details in Appendix A).

During the holding period, the *cwnd* is effectively frozen. As shown in line 18, even if valid ACKs are received, CODA-PA skips the standard window increase process in AIMD. This "freeze" behavior creates a temporal shield that prevents high-priority flows from reducing their *cwnd* prematurely while also stopping them from aggressively expanding their window to starve others. If the congestion persists after the holding timer expires, the flow then executes a delayed *cwnd* decrease (line 4) and resumes standard congestion control.

For short and periodic DLT traffic bursts, this holding mechanism effectively masks transient congestion. As illustrated in Figure 2(c), when a high-priority flow starts and collides with background traffic, its large β_j grants it a sufficient T_{hold}. This allows the flow to complete its current iteration before *cwnd* starting to decrease. Consequently, the high-priority flow maintains its throughput as if no congestion occurred. For long-running flows, such as gradient synchronization for large models, the mechanism operates in two phases. Initially, the high-priority flow utilizes its holding period to maintain its *cwnd* despite congestion signals, effectively establishing a throughput advantage. Once this holding period expires, the flow transitions into standard congestion control behavior. However, as derived in our fluid model analysis (Appendix A), the inertial effect introduced by the criticality factor β_j fundamentally alters the equilibrium point. As shown in Figure 2(d), even though both flows eventually enter the AIMD phase, the bandwidth allocation converges to a stable state proportional to $\sqrt{\beta_j}$. We prove in Appendix A.2 that:

THEOREM 3.1 (SQURE-ROOT ALLOCATION). *Algorithm 1 converges to a steady-state bandwidth allocation (B_j and B_k) for contending flows from DLT jobs j, k where:*

$$\frac{B_j}{B_k} = \sqrt{\frac{\beta_j}{\beta_k}} \tag{3}$$

We further prove the optimality of square-root allocation in Appendix A.3 that:

THEOREM 3.2 (OPTIMALITY OF SQURE-ROOT ALLOCATION). *The squre-root bandwidth allocation minimizes the cluster-wide sum of iteration times $\sum_j R_j \cdot T_{iter,j}$ under capacity constraints.*

3.3 Link-level Traffic Interleaving

We abstract the periodic DLT traffic of a job j as a tuple $(T_j, \mathcal{A}_j^l)$, where T_j denotes the iteration period, and $\mathcal{A}_j^l \subset [0, T_j)$ represents the set of active communication time intervals on link l.

To mitigate network contention, we introduce a *time shift* variable $\tau_j^l \in [0, T_j)$ for each job j on link l. This variable temporally shifts the communication phase, transforming the active interval set to:

$$\mathcal{A}_j^l(\tau_j^l) = \{(t + \tau_j^l) \mod T_j \mid t \in \mathcal{A}_j^l\} \tag{4}$$

The contention between any two jobs j_1 and j_2 on link l is quantified as the duration of the overlap between their shifted active intervals. Let $\mu(\cdot)$ denote the measure (total duration) of a time set. The pairwise contention is defined as:

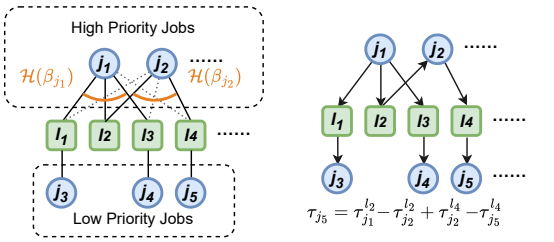
$$O_{j_1,j_2}^l(\tau_{j_1}^l, \tau_{j_2}^l) = \mu\left(\mathcal{A}_{j_1}^l(\tau_{j_1}^l) \cap \mathcal{A}_{j_2}^l(\tau_{j_2}^l)\right) \tag{5}$$

Our goal is to find the optimal time shifts that minimize the aggregate contention across all jobs on the link. The link-level traffic interleaving problem is formally stated as:

$$\min_{\{\tau_j^l\}} \sum_{1\le j_1<j_2\le|J|} O^l_{j_1,j_2}(\tau^l_{j_1},\tau^l_{j_2}) \quad s.t. \quad 0 \le \tau^l_j < T_j, \quad \forall j \in J \tag{6}$$

This optimization problem can be efficiently solved using lightweight heuristic algorithms, such as a greedy strategy that iteratively adjusts shifts to minimize local overlaps. Given its low computational complexity, we omit the detailed algorithmic steps for brevity. The resulting solution provides the specific time shifts τ^l_j for each job, which serve as the requisite input for our central traffic scheduler.

3.4 Cluster-level Traffic Interleaving



(a) Priority spanning tree **(b) Time shift propagation**

Figure 6: Spanning tree overview.

While link-level interleaving optimizes contention locally, it generates independent time shifts τ^l_j for a job j on each link l. Since a single job must operate with a globally consistent start time, these per-link shifts often conflict. To resolve the conflicts and circular dependency issues discussed in Section 2.3, we propose a priority-aware unification algorithm (Algorithm 2). This algorithm constructs a global synchronization plan by selecting a subset of link constraints that form a loop-free structure (spanning tree), weighted by the job criticality β_j derived in Section 3.2.

3.4.1 Graph Modeling and Problem Formulation. We model the cluster contention as a weighted bipartite graph $G = (J \cup L, E, w)$. The vertex sets J and L represent jobs and links, respectively. An edge $(j, l) \in E$ exists if job j uses link l, with weight $w(j, l) = O^l$ (the contention magnitude).

Our goal is to construct a spanning tree T from G that serves as the "synchronization backbone." To respect job priorities, we convert the job criticality factor β_j into a topological constraint. Specifically, we enforce a degree constraint d_j on each job vertex j, where d_j scales with β_j via mapping $\mathcal{H}(\cdot)$. This ensures that critical jobs (high β_j) act as hubs in the spanning tree, retaining more connections to preserve their local optimality, as illustrated in Figure 6(a). Consequently, these high-priority hubs serve as stable anchors during the subsequent time shift propagation process (Figure 6(b)).

Let $x_e \in \{0, 1\}$ indicate if edge e is selected in the tree T. The problem is formulated as finding a minimum weight spanning tree under degree constraints:

$$\min_{\vec{x}} \sum_{e\in E} x_e w_e \quad s.t. \quad \begin{cases} \sum_{e\in E} x_e = |J| + |L| - 1, \\ \sum_{e\in E(S)} x_e \le |S| - 1, & \forall S \subseteq J \cup L, \\ \sum_{e\in\delta(j)} x_e \le \mathcal{H}(\beta_j), & \forall j \in J. \end{cases} \tag{7}$$

3.4.2 Iterative Rounding Solution. Directly solving Eq. (7) as an Integer Program (IP) is NP-hard, therefore we employ an efficient iterative relaxation and rounding approach. We relax $x_e \in \{0, 1\}$ to $0 \le x_e \le 1$ to form a linear Program (LP). In each iteration, we solve the LP, select edges with $x_e > 0$ to form a support graph, and greedily add edges that act as bridges to our spanning tree (line 7-13 in Algorithm 2). This method effectively resolves circular dependencies by breaking cycles based on contention weights and priorities, with polynomial time complexity suitable for large-scale clusters.

3.4.3 Global Time Shift Unifying. Once the spanning tree T is constructed, we determine the global time shift τ_j for each job. We treat the tree edges as rigid constraints where the relative time difference between job j and link l must equal the locally optimized shift τ^l_j.

We arbitrarily root the tree (e.g., at a high-priority job), set its shift to 0, and propagate shifts via Breadth-First Search (BFS). For any connected pair of job j and job j' sharing link l in T, if τ_j is known, $\tau_{j'}$ is derived to preserve their relative offset:

$$\tau_{j'} = \left(\tau_j - \tau^l_j + \tau^l_{j'}\right) \mod T_{\text{LCM}} \tag{8}$$

where T_{LCM} is the least common multiple of job training iteration length. This process (line 15-19) guarantees that for all edges in the spanning tree, the pairwise contention is minimized exactly as calculated in the link-level phase. We prove the following theorem in Appendix B:

THEOREM 3.3. *Algorithm 2 converges within a polynomial number of iterations, ensuring that its overall computational complexity is polynomial.*

4 IMPLEMENTATION

We implement CODA-PA by modifying the standard Linux DCTCP congestion control module [1]. The host-side agent, written in Python, receives the job criticality factor (β_j) from

Algorithm 2: Cluster-level Traffic Interleaving

Input: Jobs J, Links L, Shifts $\{\tau_j^l\}$, Criticality $\{\beta_j\}$
1. Graph Construction: Build $G = (J \cup L, E, w)$ where $w_{jl} = O^l$.
2. Spanning Tree (Iterative Rounding):
Define degree bounds $d_j \leftarrow \mathcal{H}(\beta_j)$ for all $j \in J$
Initialize $E^T \leftarrow \emptyset$
while $|J \cup L| > 1$ **do**
  Solve LP relaxation of Eq. (7) to get $\vec{x}$
  Remove edges with $x_e = 0$ from E
  Identify edges E^* where $x_e > 0$
  for *vertex* $v \in J \cup L$ **do**
    if *v has exactly one incident edge $e = (u, v)$ in E^** **then**
      $E^T \leftarrow E^T \cup \{e\}$
      Remove v from graph; Update $d_u \leftarrow d_u - 1$
  Update graph: remove isolated nodes or relax constraints if stuck.
3. Unifying: Form Tree $T = (V, E^T)$
Pick root $r \in J$, set $\tau_r \leftarrow 0$
foreach *edge (j, l) in BFS traversal of T* **do**
  Propagate τ to maintain relative offset $\tau_j - \tau_l = \tau_j^l$
return *Global shifts $\{\tau_j\}$ for all $j \in J$*

the central scheduler and passes it to the kernel space via `setsockopt` system calls.

Inside the TCP stack, we instrumented the acknowledgment processing path to support the window holding mechanism. When ECN markings are detected, instead of triggering immediate backoff, the module calculates the holding duration T_{hold} and transitions the flow into a holding state. We hooked into the congestion avoidance function to freeze congestion window (*cwnd*) updates during this period, effectively creating the required inertia. The standard DCTCP window reduction logic is only executed if the congestion signal persists after the holding timer expires.

We implement our scheduling framework on the Ray distributed computing platform [17], leveraging its native actor system for distributed coordination. The scheduler operates as a Ray Python plugin for centralized control, interacting with an event bus actor to execute scheduling decisions based on real-time feedback. Our key innovation is fine-grained traffic management, achieved by exchanging control events (specifically `METRICS_UPDATE` and `PAUSE_SIGNAL`) between the training workers and the event bus via Ray remote calls (Listing 1).

This logic is seamlessly integrated into training workflows using PyTorch Lightning's callback system [5]. As shown in the listing, we implement a custom `CodaCallback` that hooks into the `on_train_batch_end` lifecycle event. The callback publishes trainer metrics and subsequently polls for pause instructions. Upon receiving a `PAUSE_SIGNAL`, the execution suspends immediately within the callback. This design requires no changes to the core model code while ensuring responsive link utilization with negligible overhead.

```python
import lightning.pytorch as pl
class CodaCallback(pl.Callback):
    ...
    def on_train_batch_end(self, trainer, ...):
        # publish metrics from trainer
        # via Ray actor call
        event = ControlEvent.create(
            event_type="METRICS_UPDATE",
            data=trainer.callback_metrics)
        ray.get(self.event_bus.publish.remote(event))
        # check for pause signals via
        # Ray object store
        control_events = ray.get(
            self.event_bus.get_events.remote(
            self.trainer_id,
            event_type="PAUSE_SIGNAL"
        ))
        if control_events:
            self._handle_pause(control_events[0])
```

Listing 1: Implementation of CODA's callback mechanism.

5 PERFORMANCE EVALUATION

5.1 Experimental Settings and Baselines

Testbed and Simulation Setup. We established a testbed comprising four DLT servers. To closely mimic a production DLT cluster environment, each server was configured with a 32-core Intel Xeon 6530 processor, two NVIDIA RTX A6000 GPUs, and a Mellanox ConnectX-7 100G dual-port NIC. All servers run Ubuntu 20.04, equipped with CUDA 12.4, PyTorch 2.7.0, and the Mellanox OFED 5.5-1.0.3.2 NIC drivers. The testbed network is built upon a spine-leaf topology using 100 Gbps links, interconnecting all servers and programmable switches. Specifically, two Wedge100BF-32x switches are used, with each physical switch segmented into two logical switches via VRF (Virtual Routing and Forwarding) to realize the four-node logical topology (two spine and two leaf switches) as depicted in Figure 7.

To scale our evaluation beyond physical constraints, we adopt SimAI [32], a high-fidelity discrete-event simulator. We map the centralized Ray controller described in our implementation to SimAI's scheduler module, preserving the logic

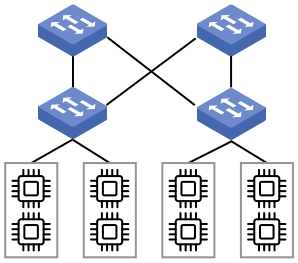


Figure 7: Testbed topology.

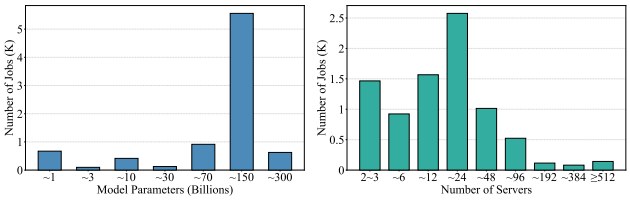


Figure 8: Model parameter size and servers distribution in our production trace.

for global coordination and traffic interleaving (*e.g.*, triggering `PAUSE_SIGNAL`s). Crucially, we extend SimAI's transport layer to implement CODA-PA by embedding the computation of dynamic priority $P(t)$ and effective marking rate α_{eff} directly into the flow control loop. This allows the simulator to enforce the priority shielding logic—where high-priority virtual flows ignore simulated congestion while non-critical flows trigger Aggressive Yield—thereby rigorously reproducing the deterministic isolation of the "virtual fast lane" at cluster scale.

Models and Simulation Workloads. In our testbed experiments, we employ a diverse set of models to cover varying computational intensities and communication patterns. Specifically, we select two dense Large Language Models (LLMs), Llama-3.2-1B and Qwen2.5-14B, alongside Qwen3-VL-4B as a representative Visual Language Model (VLM) and DeepSeek-MoE-16B to represent Mixture-of-Experts (MoE) architectures. This selection allows us to evaluate parallel strategies (including DP, FSDP, and EP) across distinct traffic characteristics, such as the heavy All-to-All communication inherent in MoE routing versus the DP AllReduce and All-Gather in dense model training.

For simulation studies, we utilize real-world traces from our production DLT cluster, comprising 8,407 training jobs after filtering out single-server DLT jobs. The dataset captures granular details including job scale, timestamps, and resource utilization metrics. As illustrated in Figure 8, the workload exhibits significant heterogeneity: the majority of jobs (6,472) involve models with 70–150 billion parameters, while the allocation of compute servers varies widely from small-scale (e.g., 1,466 jobs use 2–3 servers) to large-scale clusters (e.g., 142 jobs require 512 or more servers). These traces enable us to rigorously evaluate system performance under practical, diverse scaling conditions that are critical for studying inter-job traffic contention.

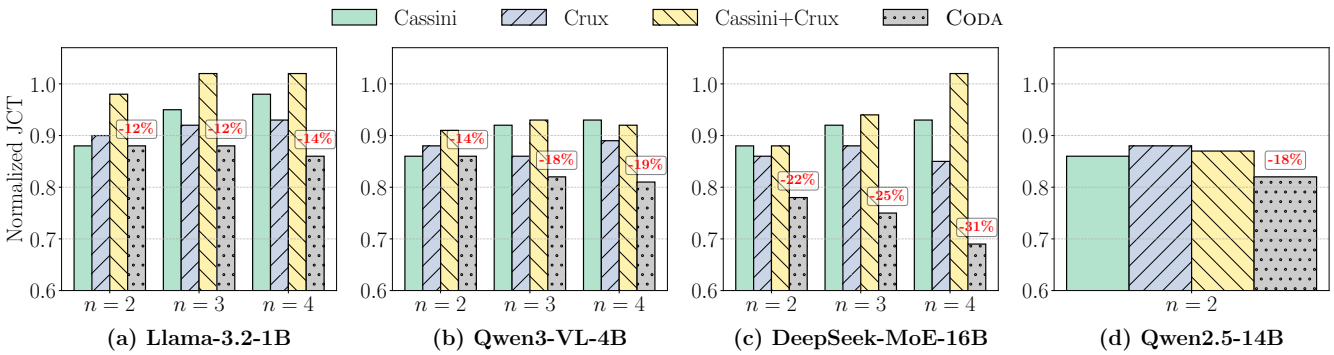
Baselines and Metrics. We evaluate CODA against two state-of-the-art baselines: Cassini [23] and Crux [4]. To align with CODA's design principles, we disable Crux's path selection. We exclude other works like MLTCP [24] (soft priority) and Symphony [3] (fine-grained interleaving) as Crux and Cassini provide the most direct and rigorous baselines for hardware-enforced isolation and global job scheduling, respectively. We additionally implement Cassini+Crux, a naive combination of two baseline approaches.

To isolate contributions of CODA's core components, we design three ablations: CODA-PA (priority control only, no traffic interleaving), CODA-TI (traffic interleaving only, with no priority control), and CODA-FULL (complete framework). We adopt the following metrics to evaluate CODA's performance:

- *Average and Tail JCT:* We measure the end-to-end Job Completion Time (JCT) for all workloads. Specifically, we report both the average JCT to demonstrate overall cluster efficiency and the 99th percentile (P99) JCT to highlight performance at the tail. This allows us to assess whether CODA effectively mitigates the starvation issues observed in static priority schemes like Crux.
- *Iteration Time Stability:* To evaluate robustness against the runtime jitter described in Section 2.3, we analyze the Cumulative Distribution Function (CDF) of training iteration durations. A steeper CDF curve indicates higher stability, confirming the system's ability to maintain precise interleaving schedules.
- *Network Congestion Indicators:* We track ECN marks and PFC pause frames per iteration. These hardware counters serve as direct indicators for network contention intensity: ECN reflects incipient congestion, while PFC frames indicate severe buffer overflows that trigger head-of-line blocking.

5.2 Testbed Results

We first deploy n identical model instances concurrently within the testbed and measure the average job completion time (JCT). Based on our setup described in Section 5.1, we select a representative set of models configured with distinct parallel strategies to evaluate CODA under varying traffic patterns: (1) Llama-3.2-1B trained with standard Data Parallelism (DP), serving as a baseline for continuous, bandwidth-intensive AllReduce traffic; (2) Qwen2.5-14B utilizing Fully



(a) Llama-3.2-1B (b) Qwen3-VL-4B (c) DeepSeek-MoE-16B (d) Qwen2.5-14B

Figure 9: Average JCT for various model architectures evaluated in the testbed.

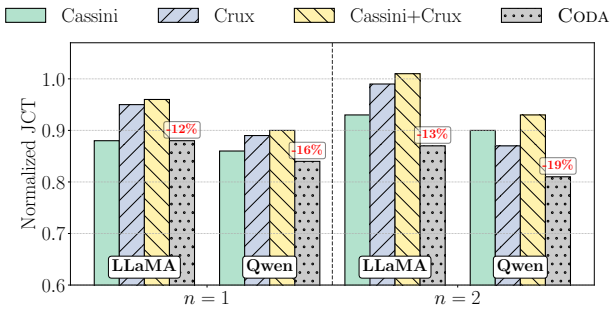


Figure 10: Average JCT under a hybrid deployment of Qwen2.5-14B and n Llama-3.2-1B jobs.

Sharded Data Parallelism (FSDP) to accommodate its large parameter set within limited GPU memory, generating complex scatter-gather communication phases; (3) DeepSeek-MoE-16B employing a hybrid of FSDP and Expert Parallelism (EP), specifically chosen to introduce bursty, irregular All-to-All traffic inherent to token routing; and (4) Qwen3-VL-4B (FSDP) to represent multimodal workloads. We evaluate Llama, Qwen3-VL, and DeepSeek-MoE with n ranging from 2 to 4 (each job utilizing 2 GPUs). Due to the high VRAM consumption, the larger Qwen2.5-14B is evaluated at $n = 2$ (each job utilizing 4 GPUs). To maximize network traffic, we enforce an anti-affinity placement strategy where each job's GPUs are distributed across different servers. Figure 9 presents the JCT reduction percentage achieved by each model under different n, normalized against the baseline performance of standard runs without any optimization.

Overall Performance. CODA consistently outperforms both Cassini and Crux across all model architectures and concurrency levels (n). For the communication-intensive DeepSeek-MoE, CODA reduces JCT by 31% at $n = 4$, whereas Cassini and Crux achieve reductions of only 7% and 15%, respectively. The naive combination (Cassini+Crux) performs poorly, often yielding lower gains than individual baselines due to conflicting control decisions (e.g., Cassini delaying a job that Crux prioritizes), aligning with our analysis in Section 2.

Scalability. As concurrency n increases, CODA demonstrates superior scalability. With the Llama-3.2 workload, CODA achieves a 12% JCT reduction at $n = 3$, increasing to 14% at $n = 4$. In contrast, Cassini's performance degrades from 12% ($n = 2$) to 2% ($n = 4$). This is because finding a conflict-free interleaving schedule becomes exponentially harder as more jobs crowd the timeline. Crux also struggles to scale, hovering around 8% reduction, as its static hardware priorities cannot distinguish urgency among multiple identical contending jobs.

Impact of Model Architecture (MoE vs. Dense). We observe distinct performance variations correlated with model architectures. Notably, CODA achieves the most significant gains on DeepSeek-MoE, reducing JCT by up to 31%. This is attributed to the unique communication patterns of Mixture-of-Experts (MoE) models. Unlike the regular AllReduce patterns in dense models (Llama/Qwen-Dense), MoE training involves Expert Parallelism (EP), generating bursty All-to-All traffic dependent on dynamic token routing.

We conclude that Cassini fails to optimize MoE effectively because the dynamic routing disrupts the strict periodicity required for pre-calculated interleaving, while Crux's static priority proves ineffective as EP traffic appears uniformly critical during the expert dispatch phase. In contrast, CODA leverages its dynamic priority mechanism to instantly detect queue buildup caused by EP bursts, creating a temporary fast lane that effectively resolves the incast congestion inherent in MoE routing.

Hybrid Co-location. We further investigate hybrid scenarios where one large Qwen2.5-14B job is co-located with n smaller Llama-3.2-1B jobs ($n = 1, 2$). Figure 10 shows the normalized JCTs. In these hybrid co-location settings, CODA maintains a 10%–20% average JCT optimization. Crucially, CODA ensures that the larger-scale Qwen job does not starve the smaller Llama jobs, a strict fairness issue observed with

Crux (where the static priority favoring the larger job penalized the smaller ones).

Hardware Counter Analysis. We analyze hardware counters collected under the high-contention DeepSeek-MoE workload ($n = 4$). As shown in Figure 11(a), CODA achieves the lowest median ECN count, reducing congestion signals by an order of magnitude compared to the baseline. Notably, while Cassini maintains a low median, it exhibits severe outliers (black dots) caused by traffic jitters, confirming the fragility of time-shift interleaving. Most critically, Figure 11(b) shows that CODA maintains near-zero PFC pause frames, whereas baselines suffer from frequent blocking. By forcing non-critical flows to yield aggressively, CODA prevents switch buffer saturation, effectively eliminating the head-of-line blocking that degrades communication performance.

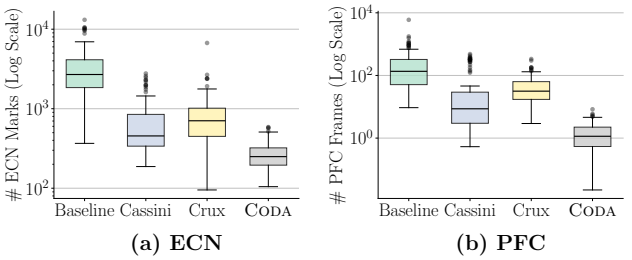


Figure 11: Network hardware counters distribution.

5.3 Simulation Results

In this subsection, we scale our evaluation to a cluster-level scope. We evaluate the contributions of CODA's key components by comparing CODA-PA (priority only), CODA-TI (interleaving only), and CODA-FULL against baselines using the production trace described in Section 5.1. We simulate two representative DLT cluster topologies: Fat-Tree and Rail-Optimized [21], each scaling to approximately 1,000 GPUs.

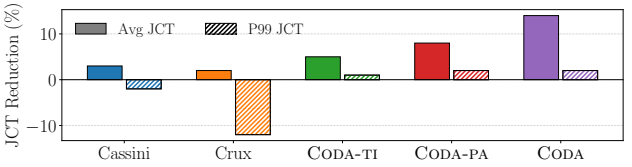


Figure 12: Average and tail JCT under fat-tree topology.

JCT Optimization: Average and Tail. Figures 12 and 13 present the JCT optimization percentages across Fat-Tree and Rail-Optimized topologies. CODA-FULL consistently achieves the highest reductions, improving average JCT by 14% and 11% respectively. Crucially, the disparity between average

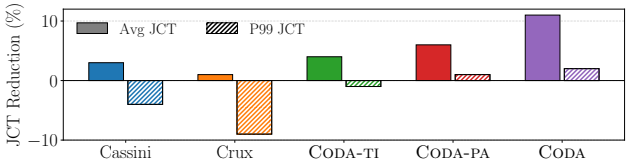


Figure 13: Average and tail JCT under rail-optimized topology.

and P99 JCT reveals significant fairness issues in baselines. While Crux achieves a slight improvement in average JCT (1%–2%), it causes severe degradation in tail performance, worsening P99 JCT by up to 12% in the Fat-Tree topology. This negative optimization confirms that static priority schemes heavily penalize low-priority jobs, leading to starvation. Similarly, Cassini suffers from tail latency degradation (-2% to -4% in P99) due to its fragility against runtime jitter. In contrast, CODA-FULL is the only method that maintains positive gains for both Average and P99 metrics (+2%), successfully preventing starvation while maximizing throughput. Among ablations, CODA-PA contributes the majority of the gains (up to 8% average), indicating that dynamic urgency is the dominant factor in stabilizing performance under heavy-tail production workloads, while CODA-TI provides supplementary optimization (up to 5%).

Iteration Time Stability. Figure 14 illustrates the CDF of training iteration times. CODA-FULL achieves a 1.5× reduction in the 95$^{\text{th}}$ percentile (P95) iteration duration compared to Cassini+Crux on both topologies. More importantly, the steepness of CODA's CDF curve indicates high predictability. Consistent with our testbed hardware counter analysis, methods relying solely on time-shift interleaving (Cassini) exhibit "long tails" in iteration time due to their fragility against runtime variance. CODA fixes this via dynamic priority shielding, ensuring that the vast majority of training iterations finish within a tight time window, providing better iteration time predictability for global interleaving.

Robustness against Failures and Stragglers. To evaluate CODA's resilience, we inject random delays into iterations with probability p (sampled from 5%–20% of duration), simulating the "stragglers" and jitter common in production environments. Figure 15 illustrates the JCT optimization as p increases from 0% to 50%.

As p increases, the performance of time-shift interleaving method (Cassini) degrades rapidly. This mirrors our testbed findings: when stragglers disrupt the precise timing required for interleaving, the pre-calculated schedule becomes invalid. CODA-TI partially mitigates this by dynamically adjusting offsets, achieving a 12% gain at $p = 50\%$ where Cassini collapses to 2%. In contrast, priority-based methods (CRUX, CODA-PA)

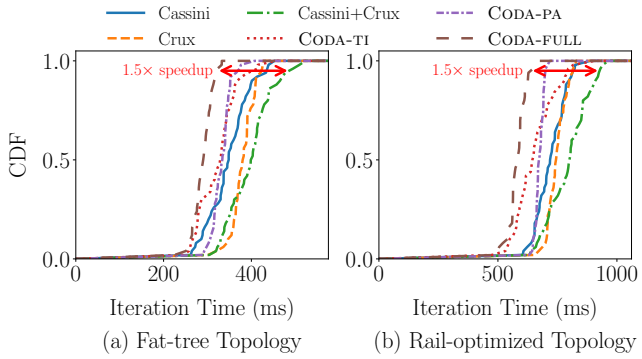


Figure 14: Iteration time CDF for each type of topology on real trace.

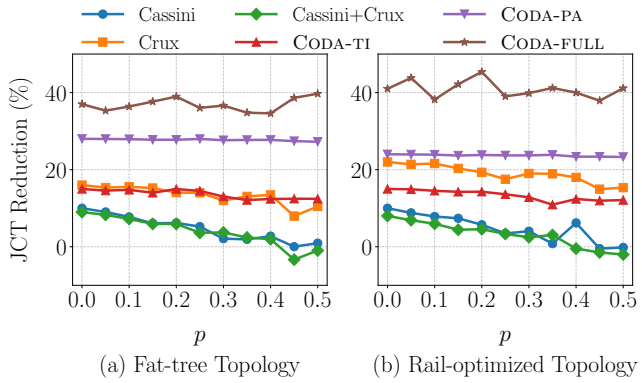


Figure 15: JCT reduction variation with failure rate p.

remain robust to timing variances but lack the peak performance of global scheduling. CODA-FULL bridges this gap, consistently maintaining at least 30% JCT optimization even under high failure rates (p = 50%). It leverages traffic interleaving for coarse-grained optimization while relying on dynamic priority assignment as a fallback execution layer to absorb runtime jitter. This synergy ensures that CODA is not only high-performing in ideal conditions but also robust for production clusters characterized by hardware heterogeneity and frequent faults.

6 RELATED WORK

Job Resource Schedulers. Job schedulers orchestrate DLT jobs across shared clusters to optimize cluster-wide efficiency and fairness. Prior works have explored various dimensions of this problem, such as designing scheduling policies to minimize job completion time or ensure fairness without prior knowledge of job duration [9, 14], and employing topology-aware placement to guarantee sharing safety in multi-tenant environments [35]. Other approaches focus on handling hardware heterogeneity [6, 19], co-adapting resource allocation with training hyperparameters [22], or leveraging elasticity to adapt to dynamic resource availability [8]. Additionally, some works extend scheduling objectives to consider multi-dimensional resource constraints beyond GPUs, such as CPU and memory bandwidth [16]. While these methods optimize resource allocation at the cluster level, they complement CODA by managing job placement and resources but lack the fine-grained mechanisms required to resolve real-time inter-job communication contention on shared links.

Intra-job Communication Schedulers. Intra-job communication schedulers optimize communication within a single DLT job to minimize latency and maximize throughput. Existing works employ various strategies, such as dynamically prioritizing tensors [15, 20], synthesizing topology-aware collective algorithms [13, 26], optimizing hierarchical traffic patterns for specific architectures like Mixture-of-Experts [12], or providing programmable abstractions for fine-grained hardware control [27]. However, since these methods operate at the job level, they do not account for cross-job contention, potentially leading to suboptimal resource utilization in multi-tenant clusters.

Inter-job Communication Schedulers. These schedulers aim to mitigate bandwidth contention among co-located jobs in multi-tenant clusters. Existing solutions primarily pursue two divergent strategies. Traffic interleaving methods (*e.g.*, Cassini [23], Symphony [3]) exploit DLT periodicity to temporally separate communication phases, but they face a fundamental trade-off between algorithmic complexity and robustness against runtime jitter. Conversely, priority assignment approaches (*e.g.*, Crux [4], MLTCP [24], PrioPlus [34]) differentiate traffic via physical or virtual priorities. However, Crux suffers from static rigidity and lack of coordination, while host-based solutions like MLTCP offer only reactive "soft" priorities that converge slowly. CODA bridges this gap by harmonizing these approaches within a unified framework, co-optimizing global time-shifting for proactive contention avoidance with dynamic priority control for robust runtime execution.

7 CONCLUSION

CODA presents a holistic solution for communication contention in multi-tenant DLT clusters, effectively bridging the gap between priority assignment and traffic interleaving. CODA introduces three key innovations: a congestion inertia mechanism to ensure schedule stability, a graph-theoretic interleaving approach to resolves complex circular dependencies, and a unified control framework to make one-shot schedules. Evaluations demonstrate that CODA significantly outperforms state-of-the-art methods, reducing average job completion time by up to 31% while maintaining superior resilience against cluster volatility.

REFERENCES

[1] Mohammad Alizadeh, Albert Greenberg, David A Maltz, Jitendra Padhye, Parveen Patel, Balaji Prabhakar, Sudipta Sengupta, and Murari Sridharan. 2010. Data center tcp (dctcp). In *Proceedings of the ACM SIGCOMM 2010 Conference*. 63–74.

[2] Mohammad Alizadeh, Adel Javanmard, and Balaji Prabhakar. 2011. Analysis of DCTCP: stability, convergence, and fairness. *ACM SIGMETRICS Performance Evaluation Review* 39, 1 (2011), 73–84.

[3] Manaf Bin-Yahya, Amir Shani, Hossein Shafieirad, Seyed Hossein Mortazavi, Chen Ying, Aaron Wang, Geng Li, and Majid Ghaderi. 2025. Symphony: Collective Coordination in Multi-Tenant GPU Clusters. In *2025 IEEE 33rd International Conference on Network Protocols (ICNP)*. IEEE, 1–13.

[4] Jiamin Cao, Yu Guan, Kun Qian, Jiaqi Gao, Wencong Xiao, Jianbo Dong, Binzhang Fu, Dennis Cai, and Ennan Zhai. 2024. Crux: Gpu-efficient communication scheduling for deep learning training. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 1–15.

[5] William Falcon and The PyTorch Lightning team. 2019. PyTorch Lightning. (March 2019). https://doi.org/10.5281/zenodo.3828935

[6] Jin Fang, Gongming Zhao, Hongli Xu, Luyao Luo, Zhen Yao, and An Xie. 2024. Non-Idle Machine-Aware Worker Placement for Efficient Distributed Training in GPU Clusters. In *2024 IEEE 32nd International Conference on Network Protocols (ICNP)*. IEEE, 1–11.

[7] Adithya Gangidi, Rui Miao, Shengbao Zheng, Sai Jayesh Bondu, Guilherme Goes, Hany Morsy, Rohit Puri, Mohammad Riftadi, Ashmitha Jeevaraj Shetty, Jingyi Yang, et al. 2024. Rdma over ethernet for distributed training at meta scale. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 57–70.

[8] Diandian Gu, Yihao Zhao, Yinmin Zhong, Yifan Xiong, Zhenhua Han, Peng Cheng, Fan Yang, Gang Huang, Xin Jin, and Xuanzhe Liu. 2023. Elasticflow: An elastic serverless training platform for distributed deep learning. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 266–280.

[9] Juncheng Gu, Mosharaf Chowdhury, Kang G Shin, Yibo Zhu, Myeongjae Jeon, Junjie Qian, Hongqiang Liu, and Chuanxiong Guo. 2019. Tiresias: A GPU cluster manager for distributed deep learning. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*. 485–500.

[10] Yimin Jiang, Yibo Zhu, Chang Lan, Bairen Yi, Yong Cui, and Chuanxiong Guo. 2020. A unified architecture for accelerating distributed DNN training in heterogeneous GPU/CPU clusters. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. 463–479.

[11] Feng Liang, Zhen Zhang, Haifeng Lu, Victor Leung, Yanyi Guo, and Xiping Hu. 2024. Communication-efficient large-scale distributed deep learning: A comprehensive survey. *arXiv preprint arXiv:2404.06114* (2024).

[12] Juncai Liu, Jessie Hui Wang, and Yimin Jiang. 2023. Janus: A unified distributed training framework for sparse mixture-of-experts models. In *Proceedings of the ACM SIGCOMM 2023 Conference*. 486–498.

[13] Xuting Liu, Behnaz Arzani, Siva Kesava Reddy Kakarla, Liangyu Zhao, Vincent Liu, Miguel Castro, Srikanth Kandula, and Luke Marshall. 2024. Rethinking machine learning collective communication as a multi-commodity flow problem. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 16–37.

[14] Kshiteej Mahajan, Arjun Balasubramanian, Arjun Singhvi, Shivaram Venkataraman, Aditya Akella, Amar Phanishayee, and Shuchi Chawla. 2020. Themis: Fair and efficient GPU cluster scheduling. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*. 289–304.

[15] Kshiteej Mahajan, Ching-Hsiang Chu, Srinivas Sridharan, and Aditya Akella. 2023. Better together: Jointly optimizing ML collective scheduling and execution planning using SYNDICATE. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. 809–824.

[16] Jayashree Mohan, Amar Phanishayee, Janardhan Kulkarni, and Vijay Chidambaram. 2022. Looking beyond GPUs for DNN scheduling on Multi-Tenant clusters. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 579–596.

[17] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I Jordan, et al. 2018. Ray: A distributed framework for emerging AI applications. In *13th USENIX symposium on operating systems design and implementation (OSDI 18)*. 561–577.

[18] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil R Devanur, Gregory R Ganger, Phillip B Gibbons, and Matei Zaharia. 2019. PipeDream: Generalized pipeline parallelism for DNN training. In *Proceedings of the 27th ACM symposium on operating systems principles*. 1–15.

[19] Deepak Narayanan, Keshav Santhanam, Fiodar Kazhamiaka, Amar Phanishayee, and Matei Zaharia. 2020. Heterogeneity-Aware cluster scheduling policies for deep learning workloads. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. 481–498.

[20] Yanghua Peng, Yibo Zhu, Yangrui Chen, Yixin Bao, Bairen Yi, Chang Lan, Chuan Wu, and Chuanxiong Guo. 2019. A generic communication scheduler for distributed DNN training acceleration. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*. 16–29.

[21] Kun Qian, Yongqing Xi, Jiamin Cao, Jiaqi Gao, Yichi Xu, Yu Guan, Binzhang Fu, Xuemei Shi, Fangbo Zhu, Rui Miao, et al. 2024. Alibaba hpn: A data center network for large language model training. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 691–706.

[22] Aurick Qiao, Sang Keun Choe, Suhas Jayaram Subramanya, Willie Neiswanger, Qirong Ho, Hao Zhang, Gregory R Ganger, and Eric P Xing. 2021. Pollux: Co-adaptive cluster scheduling for goodput-optimized deep learning. In *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*.

[23] Sudarsanan Rajasekaran, Manya Ghobadi, and Aditya Akella. 2024. CASSINI: Network-Aware Job Scheduling in Machine Learning Clusters. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. 1403–1420.

[24] Sudarsanan Rajasekaran, Sanjoli Narang, Anton A Zabreyko, and Manya Ghobadi. 2024. Mltcp: A distributed technique to approximate centralized flow scheduling for machine learning. In *Proceedings of the 23rd ACM Workshop on Hot Topics in Networks*. 167–176.

[25] Samyam Rajbhandari, Conglong Li, Zhewei Yao, Minjia Zhang, Reza Yazdani Aminabadi, Ammar Ahmad Awan, Jeff Rasley, and Yuxiong He. 2022. Deepspeed-moe: Advancing mixture-of-experts inference and training to power next-generation ai scale. In *International conference on machine learning*. PMLR, 18332–18346.

[26] Aashaka Shah, Vijay Chidambaram, Meghan Cowan, Saeed Maleki, Madan Musuvathi, Todd Mytkowicz, Jacob Nelson, Olli Saarikivi, and Rachee Singh. 2023. TACCL: Guiding collective algorithm synthesis using communication sketches. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. 593–612.

[27] Aashaka Shah, Abhinav Jangda, Binyang Li, Caio Rocha, Changho Hwang, Jithin Jose, Madan Musuvathi, Olli Saarikivi, Peng Cheng, Qinghua Zhou, Roshan Dathathri, Saeed Maleki, and Ziyue Yang. 2025. MSCCL++: Rethinking GPU Communication Abstractions for Cutting-edge AI Applications. (2025). arXiv:cs.DC/2504.09014 https://arxiv.org/abs/2504.09014

[28] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053* (2019).

[29] Mohit Singh and Lap Chi Lau. 2015. Approximating minimum bounded degree spanning trees to within one of optimal. *Journal of the ACM (JACM)* 62, 1 (2015), 1–19.

[30] Zhenheng Tang, Shaohuai Shi, Wei Wang, Bo Li, and Xiaowen Chu. 2020. Communication-efficient distributed deep learning: A comprehensive survey. *arXiv preprint arXiv:2003.06307* (2020).

[31] Kimi Team, Yifan Bai, Yiping Bao, Guanduo Chen, Jiahao Chen, Ningxin Chen, Ruijue Chen, Yanru Chen, Yuankun Chen, Yutian Chen, et al. 2025. Kimi k2: Open agentic intelligence. *arXiv preprint arXiv:2507.20534* (2025).

[32] Xizheng Wang, Qingxu Li, Yichi Xu, Gang Lu, Dan Li, Li Chen, Heyang Zhou, Linkang Zheng, Sen Zhang, Yikai Zhu, et al. 2025. {SimAI}: Unifying Architecture Design and Performance Tuning for {Large-Scale} Large Language Model Training with Scalability and Precision. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*. 541–558.

[33] Zhiyi Yao, Pengbo Hu, Congcong Miao, Xuya Jia, Zuning Liang, Yuedong Xu, Chunzhi He, Hao Lu, Mingzhuo Chen, Xiang Li, et al. 2025. Holmes: Localizing Irregularities in LLM Training with Mega-scale GPU Clusters. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*. 523–540.

[34] Zhaochen Zhang, Feiyang Xue, Keqiang He, Zhimeng Yin, Gianni Antichi, Jiaqi Gao, Yizhi Wang, Rui Ning, Haixin Nan, Xu Zhang, et al. 2025. Enabling Virtual Priority in Data Center Congestion Control. In *Proceedings of the Twentieth European Conference on Computer Systems*. 396–412.

[35] Hanyu Zhao, Zhenhua Han, Zhi Yang, Quanlu Zhang, Fan Yang, Lidong Zhou, Mao Yang, Francis CM Lau, Yuqi Wang, Yifan Xiong, et al. 2020. HiveD: Sharing a GPU cluster for deep learning with guarantees. In *14th USENIX symposium on operating systems design and implementation (OSDI 20)*. 515–532.

A FLUID MODEL ANALYSIS OF WINDOW HOLDING MECHANISM

In this section, we analyze the window holding mechanism using a fluid model framework based on the DCTCP analysis [1, 2]. We first formulate the system dynamics using delay-differential equations, then solve for the steady-state bandwidth allocation, and finally demonstrate that this allocation minimizes the total iteration time for distributed training workloads.

A.1 Fluid Model Formulation

We consider N long-lived flows sharing a single bottleneck link with capacity C. Let $W_j(t)$ be the congestion window size of flow j, $R(t)$ be the round-trip time, and $q(t)$ be the instantaneous queue size. The switch marks packets when $q(t) > K$.

The standard DCTCP fluid model describes the evolution of the window $W_j(t)$ and the exponential weighted moving average (EWMA) of the marking fraction $\alpha_j(t)$ as follows:

$$\frac{dW_j}{dt} = \frac{1}{R(t)} - \frac{W_j(t)\alpha_j(t)}{2R(t)} p(t - R^*), \tag{9}$$

$$\frac{d\alpha_j}{dt} = \frac{g}{R(t)} \big(p(t - R^*) - \alpha_j(t)\big), \tag{10}$$

$$\frac{dq}{dt} = \sum_{j=1}^{N} \frac{W_j(t)}{R(t)} - C, \tag{11}$$

where $p(t) = \mathbf{1}_{\{q(t)>K\}}$ is the marking process and R^* represents the propagation delay plus the queuing delay at the threshold K.

Our proposed mechanism modifies the window evolution by introducing a holding period. Upon receiving a congestion signal, flow j with criticality β_j delays the window reduction for a duration $T_{\text{hold},j}$. This creates a shielding effect where a subset of marking signals is effectively ignored. We define the effective marking process $p_j^{\text{eff}}(t)$ for flow j as the original marking process gated by the holding logic.

To incorporate this into the fluid model, we approximate the discrete holding behavior with its time-averaged effect. Assuming the system operates in a regime where congestion epochs occur periodically with duration ΔT, and given a holding duration $T_{\text{hold},j} < \Delta T$, the fraction of time during which congestion signals are effective is $1 - T_{\text{hold},j}/\Delta T$. Since the congestion epoch duration in steady state approximates the RTT ($\Delta T \approx R^*$), and the holding time is set as $T_{\text{hold},j} = \kappa\beta_j R^*$, we introduce an effectiveness factor γ_j:

$$\gamma_j = \max\left(0, 1 - \frac{T_{\text{hold},j}}{R^*}\right) = \max\left(0, 1 - \kappa\beta_j\right). \tag{12}$$

Incorporating γ_j into Eq. (9), the modified window dynamics become:

$$\frac{dW_j}{dt} = \frac{1}{R(t)} - \frac{W_j(t)\alpha_j(t)}{2R(t)} \gamma_j p(t - R^*). \tag{13}$$

This equation reflects that flow j responds to congestion signals with a reduced probability proportional to γ_j.

We normalize the system variables to simplify the analysis. Let $\bar{W} = (Cd + K)/N$, $\tilde{W}_j = W_j/\bar{W}$, and $\tilde{q} = (q - K)/(CR^*)$. The normalized dynamics are:

$$\frac{d\tilde{W}_j}{dt} = \frac{1}{1 + \tilde{q}(t)} \left[1 - \frac{\tilde{W}_j(t)\alpha_j(t)}{2} \gamma_j \tilde{p}(t-1)\right], \tag{14}$$

$$\frac{d\alpha_j}{dt} = \frac{g}{1 + \tilde{q}(t)} \big(\tilde{p}(t-1) - \alpha_j(t)\big), \tag{15}$$

$$\frac{d\tilde{q}}{dt} = \frac{1}{\bar{W}} \sum_{k=1}^{N} \frac{\tilde{W}_k(t)}{1 + \tilde{q}(t)} - 1. \tag{16}$$

A.2 Steady-State Bandwidth Allocation

We analyze the steady-state bandwidth allocation by examining the equilibrium of the window evolution process. While the fluid model describes the smooth trajectory, the bandwidth allocation in an AIMD (Additive Increase Multiplicative Decrease) system is determined by the balance between the constant increase rate and the stochastic decrease events.

In the proposed mechanism, flow j increases its window linearly by 1 packet per RTT. Upon congestion, the standard DCTCP decreases the window proportional to $\alpha/2$. However, the window holding mechanism effectively scales this decrease probability. The *effective* decrease coefficient for flow j, denoted as δ_j, is the product of the congestion estimate and the holding factor:

$$\delta_j = \frac{\alpha^*}{2} \cdot \gamma_j = \frac{\alpha^*}{2}(1 - \kappa\beta_j). \tag{17}$$

According to the macroscopic equilibrium law for AIMD flows, the steady-state window size W_j^* is inversely proportional to the square root of its effective decrease coefficient δ_j. Specifically, $W_j^* \approx \sqrt{2/\delta_j}$. Substituting δ_j, we obtain:

$$W_j^* \propto \sqrt{\frac{1}{\gamma_j}} = \frac{1}{\sqrt{1 - \kappa\beta_j}}. \tag{18}$$

This relationship reveals that the window holding mechanism introduces a non-linear gain for high-criticality flows. To compare this with the target square-root allocation derived in Appendix A.3, we examine the first-order behavior of this allocation.

Using the Taylor series expansion for small holding factors ($\kappa\beta_j \ll 1$):

$$W_j^* \propto (1 - \kappa\beta_j)^{-1/2} \approx 1 + \frac{1}{2}\kappa\beta_j. \tag{19}$$

Consider the ideal square-root allocation where bandwidth scales with $\sqrt{1 + \kappa\beta_j}$ (a proxy for criticality). Its expansion is similarly:

$$\sqrt{1 + \kappa\beta_j} \approx 1 + \frac{1}{2}\kappa\beta_j. \tag{20}$$

Since the first-order terms match, the mechanism $W_j^* \propto 1/\sqrt{1 - \kappa\beta_j}$ serves as a mathematically robust approximation for square-root differentiation $\sqrt{\beta_j}$ in the operating region. Consequently, the allocation ratio between two flows satisfies:

$$\frac{B_j^*}{B_k^*} = \sqrt{\frac{\gamma_k}{\gamma_j}} \approx \sqrt{\frac{\beta_j}{\beta_k}}, \tag{21}$$

assuming the criticality parameters are scaled such that $1/\gamma_j$ maps to the magnitude of β_j. This proves that modifying the effective decrease frequency via window holding naturally converges to the optimal square-root bandwidth distribution required for distributed training.

A.3 Optimality of the Allocation

We next show that the bandwidth allocation ratio derived above is optimal for minimizing the total time to complete an iteration in distributed training.

Consider n concurrent jobs. Job j is characterized by its communication volume V_j, computation time τ_j^{comp}, and a remaining number of iterations R_j. The iteration time is determined by the bottleneck between computation and communication: $T_{\text{iter},j}(B_j) = \max(\tau_j^{\text{comp}}, V_j/B_j)$, assuming perfect scaling for simplicity ($\eta = 1$).

We focus on the regime where jobs are communication-bound, as this is where bandwidth allocation impacts performance. The objective is to minimize the sum of remaining processing times:

$$\min_{\{B_j\}} \sum_{j=1}^{n} R_j \frac{V_j}{B_j} \quad \text{s.t.} \quad \sum_{j=1}^{n} B_j = C,\ B_j > 0. \tag{22}$$

We solve this optimization problem using the method of Lagrange multipliers. The Lagrangian is:

$$\mathcal{L} = \sum_{j=1}^{n} \frac{R_j V_j}{B_j} + \lambda \left(\sum_{j=1}^{n} B_j - C \right). \tag{23}$$

Taking the partial derivative with respect to B_j and setting it to zero:

$$\frac{\partial \mathcal{L}}{\partial B_j} = -\frac{R_j V_j}{B_j^2} + \lambda = 0 \implies B_j = \sqrt{\frac{R_j V_j}{\lambda}}. \tag{24}$$

The optimal bandwidth for job j is proportional to $\sqrt{R_j V_j}$. Since the remaining iterations R_j decrease uniformly across synchronized jobs, the relative priority is determined by the static job characteristics. By defining the criticality β_j proportional to the communication intensity (e.g., V_j/τ_j^{comp}), the optimal ratio between any two jobs becomes:

$$\frac{B_j^*}{B_k^*} = \sqrt{\frac{\beta_j}{\beta_k}}. \tag{25}$$

Comparing this result with the steady-state analysis, we conclude that the window holding mechanism drives the system toward the theoretically optimal bandwidth allocation.

B COMPLEXITY OF INTERLEAVING

Polynomial Complexity of Interleaving. We analyze the computational complexity of Algorithm 2 by examining its three main stages: graph construction, spanning tree construction (iterative rounding), and time shift unifying. Let $n = |J| + |L|$ denote the total number of vertices (jobs and links) and $m = |E|$ denote the number of edges in the bipartite graph.

Graph Construction and Unifying (Step 1 & 3). Step 1 constructs the bipartite graph, which involves iterating

through job-link relationships, taking $O(m)$ time. Step 3 performs a Breadth-First Search (BFS) traversal on the constructed spanning tree to propagate time shifts. BFS typically runs in $O(n)$ time since a tree has $n-1$ edges. Both steps are trivially polynomial.

Spanning Tree Construction (Step 2). This step employs an iterative rounding technique based on Linear Programming (LP). The complexity depends on two factors: the time required to solve the LP relaxation and the number of iterations.

The LP formulation (Eq. 7) has a polynomial number of variables and constraints (excluding the subtour elimination constraints which can be separated in polynomial time). Standard algorithms such as the Ellipsoid method or Interior Point methods can solve such LPs in polynomial time, denoted as T_{LP}. The convergence of the iterative loop relies on the well-established properties of *extreme point solutions* for bounded degree spanning tree polyhedra. Theoretical results [29] guarantee that for any extreme point solution x^* with support $E^* = \{e \mid x_e^* > 0\}$, one of the following conditions must hold:

- There exists an edge e with $x_e^* = 0$ (which is removed).
- There exists a vertex v with degree 1 in the support graph (V, E^*) (i.e., a leaf node).
- (In generalized cases) There exists a vertex where the degree constraint can be relaxed.

In our algorithm, line 11 specifically targets vertices with degree 1 in the support graph. The theory guarantees that as long as the graph is not empty, such a vertex (or a redundant constraint) always exists. Consequently, in each iteration of the `while` loop, the algorithm either removes a subset of edges (where $x_e = 0$) or permanently fixes an edge to the spanning tree and removes a vertex v from the problem instance.

Since at least one vertex or edge is removed in every iteration, the loop is bounded by $O(n+m)$ iterations. The total time complexity is bounded by $O((n+m) \cdot T_{LP})$. Since both n, m, and T_{LP} are polynomial with respect to the input size, Algorithm 2 terminates in polynomial time. □

Scheduling Overhead and Scalability. We evaluate the scalability of Coda's centralized scheduler by measuring the decision latency as the cluster load increases. Figure 16 presents the scheduling overhead (average and range) as the number of concurrent jobs scales from 50 to 1,000.

The results demonstrate that the overhead grows sublinearly with the number of jobs. Even under a heavy load of 1,000 concurrent jobs, the average scheduling latency remains around 13.8 seconds, with the worst-case tail latency comfortably below 20 seconds. This efficiency is attributed to our one-shot optimization strategy, which avoids the prohibitive computational cost of iterative rescheduling. Given that DLT jobs typically run for hours or days, this minimal startup latency (less than 0.1% of typical job duration) confirms that Coda's centralized control plane incurs negligible overhead, making it highly suitable for large-scale production deployments.

Figure 16: Scheduling overhead with different number of DLT jobs in the cluster.